Catching Cover-Ups on Windows: Cross-Checking Independent Records to Detect Timestamp Tampering

Captain Scott Russell
CPRE 5360 Computer and Network Forensics, Iowa State University
Draft of the proposed solution and evaluation plan, November 1, 2026

***Abstract*—A person hiding their tracks on a Windows computer can change the dates on a file in seconds, and the dates Windows shows will then say whatever they chose. But Windows records the same activity in several other places, written by different parts of the system for different reasons. Changing all of them so that they still agree is much harder. This paper proposes the Artifact Disagreement Scorer (ADS), a tool that lines up six of these independent records, checks them against each other with eleven rules, notices when records have been deleted, and combines the results into one score per file with a plain-language reason for every flag. Earlier work showed each record is unreliable alone but never measured whether combining them helps. The evaluation plan measures that: on Windows 11 machines in ordinary use, with a written answer key of every change made, does the combined score catch more tampering than any single check at the same false alarm rate?**

***Index Terms*—Digital forensics, anti-forensics, timestamp manipulation, NTFS, tool validation, triage**

I. INTRODUCTION

Dates are central to most investigations. They place a person at a keyboard, order events, and separate an old document from a new one. On Windows, the dates a user sees for each file (created, modified, accessed) are stored in a structure that any program with ordinary rights can rewrite. Changing them, often called *timestomping*, is a documented anti-forensic technique [4] that takes a single command.

Windows also keeps other records of what happened to a file, and none of them was designed as evidence. The NTFS file system keeps a second, harder-to-reach copy of the dates. Its change journal logs file creations and changes. The Windows shell writes shortcut (LNK) files when a document is opened. Prefetch files record when programs ran. Event logs record logons, shutdowns, and clock changes. A person covering their tracks may alter one or two of these; keeping all of them consistent is much harder. A cover-up therefore tends to leave records that *disagree* (Fig. 1).

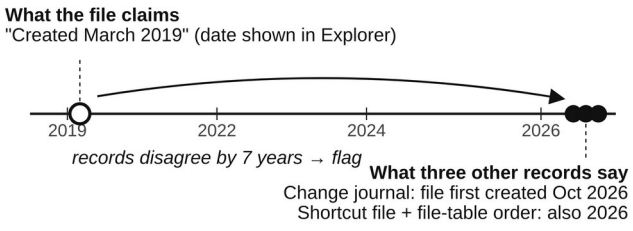


Fig. 1. What a disagreement looks like. The file's visible date was set back to 2019, but three independent records place its creation in 2026.

Existing tools mostly leave these comparisons to the examiner. Commercial suites such as EnCase and FTK display the records but leave the analyst to notice that two of them disagree and decide whether it matters. Specialized tools automate individual checks, but each check alone is weak, because normal activity (installers, archive tools, cloud sync, antivirus scans) also produces odd dates.

This paper asks one question: *does measuring how much several independent records disagree detect tampering more reliably than any single check?* Section II reviews the prior work this builds on. Section III describes the proposed tool. Section IV gives the plan to test it.

II. BACKGROUND AND RELATED WORK

Terms. NTFS stores four dates for each file in its *Master File Table* (MFT): Created, Modified, Accessed, and Entry-changed (when the file's record itself last changed). It stores them twice. The *Standard Information* copy (SI) is what Explorer shows and what programs can set. The *File Name* copy (FN) is maintained by the file system and cannot be set through the normal programming interface. This paper writes SI-C for the SI created date, FN-C for the FN created date, and so on.

Single-record checks. Early work set out rules that NTFS dates obey in normal use [3] and noted that SI dates earlier than FN dates [5], or SI dates with an all-zero fraction of a second [6], suggest tampering. Tools now automate some of these; Magnet AXIOM, for example, reports SI and FN mismatches [10], which is one check (R6 below) on its own. Timeline tools such as Plaso merge records by time but do not test whether they agree. Recent work studies NTFS dates in more depth [11], [12]. Willassen [7] and Casey [8] showed that the order in which records are allocated on disk, like layers of rock, can reveal a file that claims to be older than its neighbors.

Combining records. Palmbach and Breitinger [1] are the closest work. They showed that the change journal, LNK files, Prefetch, and event logs all carry traces of timestamp changes, concluded that none is reliable alone, and proposed five consistency rules across them. They recommend combining sources but did not measure whether combining helps, and their test used four files on an otherwise unused system, so false alarms on normal activity were never counted. Vanini, Hargreaves, and Breitinger [2] proposed seven factors for rating how hard a record is to tamper with, but their framework rates the record type, not whether a given system shows that tampering happened.

The gap. No prior work measures, on a system in ordinary use, whether combining cross-record checks beats the best single check. That is the gap this paper addresses.

III. PROPOSED SOLUTION

A. Overview and Design Goals

The Artifact Disagreement Scorer (ADS) reads six independent records from an image of a Windows drive, lines them up file by file, applies a set of consistency rules, and combines the results into a disagreement score for each file and a warning level for the system as a whole (Fig. 2). Four goals shape the design:

1. **Combine, do not trust one check.** No single rule decides. A file is flagged when the weight of independent disagreement crosses a threshold, so normal noise in one record does not raise an alert on its own.
2. **Treat a missing record as a clue.** Deleting a record is often easier than altering it convincingly [1]. ADS reports cleared or missing records instead of silently skipping the rules that need them.
3. **Explain every flag.** Every score breaks down into the rules that fired and the values they compared, written in plain language, so an examiner can state and defend exactly why a file was flagged.
4. **Reuse trusted parsers.** Reading these records is a solved problem. ADS uses widely tested open tools for that and contributes only the cross-checking and scoring layer, which is where existing tools stop.

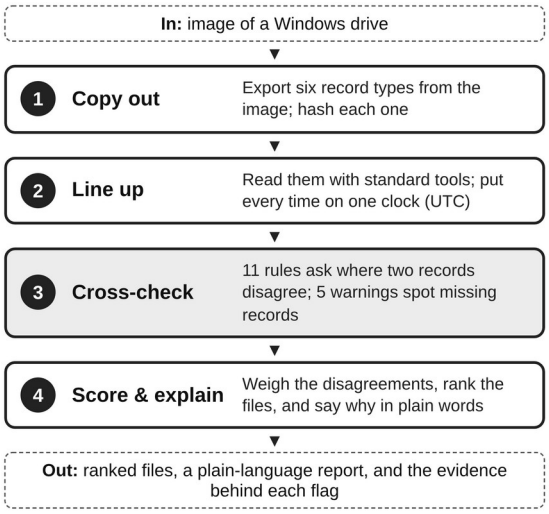


Fig. 2. The ADS pipeline. Each stage keeps a hash of its input, so the chain of custody reaches every intermediate file.

Scope. One Windows release (Windows 11 24H2), NTFS system drives, changes made from inside the running system, and test machines I build myself. Changes made by editing the raw disk offline bypass the journals and are out of scope.

B. Records Used and How They Are Linked

Table I lists the six records. They were chosen because each is written by a different part of Windows, which is what makes their agreement meaningful.

TABLE I
RECORDS USED BY ADS

Record	Written by	What ADS uses	Linked by
MFT (SI and FN)	File system	Eight dates, record number	File ID
Change journal ($J)	File system	Create and date-change entries	File ID
$LogFile	File system	Before and after copies of changes	File ID
Shortcut (LNK) files	Windows shell	Own dates; copy of target's dates	Path
Prefetch	Windows	Program name, last 8 run times	Path, time
Event logs	Event Log service	Logons, log clearing, clock changes	Time

The file ID (MFT record number plus a reuse counter) is stored directly in journal entries, so those links are exact. Shortcut and Prefetch files name their targets by path, so those links use a cleaned-up full path, and the reuse counter rejects matches to a recycled record. Event log entries name no file and are linked by time.

C. Consistency Rules

ADS starts from the five rules in [1] and adds six more. Each rule gives one of three answers for a file: *consistent*, *violated*, or *cannot check* when a needed record is missing. Table II lists them.

TABLE II
CONSISTENCY RULES (C = CREATED, A = ACCESSED, E = ENTRY CHANGED)

ID	The rule checks that...	From
R1	SI-E is not earlier than the journal's last date-change entry	[1]
R2	A shortcut's SI-E matches its target's SI-E	[1]
R3	A shortcut was not created before its target	[1]
R4	A shortcut's SI-A is not after its target's SI-A	[1]
R5	SI-C and SI-M fall inside a logged-on session	[1]
R6	SI-C is not earlier than FN-C	[5]
R7	SI dates do not have an all-zero fraction of a second	[6]
R8	The journal's file-created entry agrees with SI-C	New
R9	The target date copied inside a shortcut agrees with SI-C	New
R10	A date change does not line up with an unusual program run	[1] ext.
R11	SI-C is plausible next to neighboring MFT records	[7], [8]

R8 and R9 are the main new rules. The journal writes a file-created entry when a file is made, and a later change to the visible dates does not touch it, so a created entry dated well after the claimed creation date is strong evidence of backdating while the journal still holds it. Likewise, every shortcut keeps a copy of its target's dates from when the link was last updated; a creation date that has since moved is suspicious even when the shortcut's own dates look normal. R10 generalizes an observation in [1] that date changes lined up with Prefetch runs of the program that made them. R11 applies the "layers of rock" idea [7], [8]; it is expected to be the noisiest rule and is

included to test whether a weak signal still helps once combined.

Rules that compare times allow a small tolerance τ, because related records are not written at exactly the same instant: two seconds for journal comparisons and sixty seconds for sessions to start, then tuned on clean data.

D. Warnings About Missing Records

A file-level rule cannot fire if its record is gone. To keep that absence visible, ADS runs five system-level checks: **S1** an event log was cleared (System 104, Security 1102); **S2** the change journal was deleted (Application 3079) or starts suspiciously late; **S3** event logs have gaps in their record numbers; **S4** the system clock was changed (Security 4616); and **S5** files that were clearly opened have no shortcut, or programs that clearly ran have no Prefetch file. These checks do not prove any file was altered. They mark every "consistent" answer that rests on a damaged record as low-confidence, so the examiner sees that the evidence has been degraded instead of a misleadingly clean result.

E. Scoring

The score rewards agreement among independent rules, discounts rules that fire often on normal activity, and gives more weight to records that are hard to tamper with:

$$D(f) = \Sigma_r \, w_r \, v_r(f) \;/\; \Sigma_r \, w_r \qquad (1)$$

Here $v_r(f)$ is 1 if rule r is violated for file f and 0 if consistent; rules that cannot be checked are left out of both sums, and the number of rules that could be checked is reported beside D(f), so a score based on one rule is never confused with one based on six.

Each weight w_r multiplies two terms. The first is $\ln(1/p_r)$, where p_r is how often rule r fires on a clean machine: a rule that fires on 2% of normal files counts for more than one that fires on 20%. The second turns the seven tamper-resistance factors of [2] into a number from 1 to 3, so records a user cannot easily reach (the journal, the FN dates) count for more than records a user can edit or delete (shortcuts, Prefetch). To my knowledge this is the first use of [2] as a numeric input to detection.

A file is flagged when D(f) exceeds a threshold θ chosen on separate calibration data to meet a target false alarm rate. A weighted sum was chosen over a trained classifier because a lab experiment has few tampered files, which would let a trained model overfit, and because a sum keeps every flag explainable. Finally, flagged files whose evidence falls within five minutes of each other are grouped with any program runs and warnings in that window, turning a list of files into a candidate event to investigate.

F. Implementation and Usability

ADS is written in Python 3 with only the standard library, so it runs on any computer with Python installed. Parsing uses Eric Zimmerman's MFTECmd, PECmd, LECmd, and EvtxECmd [9]; ADS loads their output into one SQLite database with every time stored in UTC at full precision and a SHA-256 hash of every input. Each rule is a separate function that writes one answer per file together with the values it compared, so rules can be added without touching the scorer. The same code produces each single-rule baseline ("R1 only," "R2 only," and so on) from identical inputs, which is the comparison the evaluation needs.

Because findings often go to investigators, lawyers, and commanders who are not forensic specialists, ADS is built to be usable and readable by them. It opens in a simple point-and-click window, and its main output is a single web page that explains each flagged file in plain sentences ("the date shown says 2019, but the change journal records the file being created in October 2026") with the technical values folded away beneath. A ranked spreadsheet and the raw results are written alongside for examiners. The full pipeline is already implemented and passes automated tests on synthetic cases.

IV. EVALUATION PLAN

A. Research Questions

- **RQ1. Better than single checks?** At a fixed false alarm rate, does D(f) catch more tampered files than any single rule R1 to R11?
- **RQ2. How noisy is normal use?** How often does each rule, and the combined score, fire on files no one tampered with? This is the measurement missing from [1].
- **RQ3. Still works when records are deleted?** Does ADS still detect tampering after the journal, shortcuts, Prefetch, or event logs are cleared, and do the warnings S1 to S5 report the damage?
- **RQ4. Practical and understandable?** Can the report alone explain each flag, and does the tool run in reasonable time on a full image?

B. Test Machines and Answer Key

All test systems are Windows 11 24H2 virtual machines with default settings for Prefetch, event log sizes, and the change journal, recorded so the setup can be repeated. The correct answer must be known before ADS runs. A script inside the virtual machine performs each planned change and writes an answer-key line for it (file, file ID, category, and time); the time from the host computer is also recorded, so a scenario that changes the guest's clock cannot corrupt the key. A file is *tampered* if it is in the answer key and *clean* otherwise, a conservative choice that may slightly overstate false alarms. Each scenario is applied to a fresh copy of a common base snapshot so scenarios cannot contaminate each other.

C. Tampering Scenarios

Scenarios are drawn from techniques documented in [1] and MITRE ATT&CK T1070 [4]. Exact tools and versions will be listed in the final paper.

TABLE III
SCENARIO CATEGORIES AND THE CHECKS EXPECTED TO RESPOND

ID	Scenario	Expected to respond
T1	Change SI dates through Windows	R1, R5, R7, R8, R9, R10
T2	Change SI and FN dates together	R1, R5, R8, R9, R11
T3	Make a new file look old	R6, R8, R9, R11
T4	Clear event logs	S1, S3
T5	Delete the change journal	S2
T6	Delete shortcuts and Prefetch	S5
T7	Change the system clock	S4, R5, R11
T8	T1 or T2, then any of T4 to T6	All of the above

T1 to T3 alter files and are scored per file; T4 to T7 damage the evidence and are scored per system. T8 models a careful adversary who alters files and then removes the records most likely to expose them, the main test of RQ3. Each category is repeated on documents, images, and programs in several folders.

D. Data Sets

Four sets of images are kept strictly apart so tuning never sees test data. **Baseline (B):** one machine used normally for at least two weeks (browsing, editing, installing and updating software, unzipping archives, cloud sync, Windows updates and antivirus scans) with no tampering; it supplies each p_r and answers RQ2. **Calibration (C):** a few copies with some scenarios applied, used only to set τ and θ. **Test (T):** separate copies with every scenario, the only source of reported detection results. **External (E):** the course's project images, which I did not build; with no answer key they give only an upper bound on false alarms and check that the parsers cope with unfamiliar data.

E. Measures and Experiments

For each file ADS is compared with the answer key to give the detection rate (share of tampered files flagged), the false alarm rate (share of clean files flagged, also given as a raw count per image, since that is what an examiner experiences), and precision. Sweeping θ gives a receiver operating characteristic (ROC) curve and its area (AUC) for ADS and for each single rule, so they can be compared without choosing one threshold first. Detection at false alarm rates of 1% and 0.1% answers RQ1 directly. Coverage, grouping accuracy, warning accuracy on T4 to T7, and run time are also reported.

E1 compares every single rule with ADS on the test set (RQ1). **E2** reports each rule's firing rate on the baseline with the most common harmless causes (RQ2). **E3** reruns ADS with each record type removed in turn and on the T8 images (RQ3). **E4** compares equal weights, the first weight term alone, and both terms, to test whether turning [2] into numbers helps. **E5** compares the weighted sum with a logistic regression trained on the calibration set. **E6** checks, for a sample of flags, that the report alone explains each one (RQ4).

F. Success Criteria

The proposal is supported if, on the test set: (1) ADS has a higher detection rate than the best single rule at both 1% and 0.1% false alarm rates; (2) ADS has a larger AUC than any single rule; (3) on every T8 image ADS either flags the altered files or a warning reports the removal, so no scenario produces a clean result with no warning; (4) full weighting does at least as well as equal weighting; and (5) the parser cross-check agrees on at least 99% of sampled records or every disagreement is explained. If ADS does not beat the best single rule, that is itself a finding: the advice in [1] to combine records would not hold once normal activity is included, and E2 would show why.

G. Threats to Validity

The same person built the test data and the rules, which can bias results toward detection; holding out the test set, fixing θ on calibration data only, and running on the external images reduce this risk. Two weeks on one machine cannot represent every workload, so firing rates are estimates for this platform. The adversary is assumed not to know the rules; offline disk editing and an adversary who keeps every record consistent are out of scope. The number of tampered files is small, so results are given as counts with confidence intervals.

H. Status and Schedule

The tool and the scenario scripts are already written and tested on synthetic cases, so the time goes to the step that cannot be rushed: ordinary use of the baseline machine (to Nov 15), then scenarios and calibration (Nov 16–22), experiments and course images (Nov 23–29), and the final paper (by Dec 6).

REFERENCES

[1] D. Palmbach and F. Breitinger, "Artifacts for detecting timestamp manipulation in NTFS on Windows and their reliability," *Forensic Sci. Int.: Digit. Investig.*, vol. 32, 300920, 2020.

[2] C. Vanini, C. Hargreaves, and F. Breitinger, "Evaluating tamper resistance of digital forensic artifacts during event reconstruction," arXiv:2412.12814, 2024.

[3] K.-P. Chow, F. Y. Law, M. Y. Kwan, and P. K. Lai, "The rules of time on NTFS file system," in *Proc. 2nd Int. Workshop Systematic Approaches to Digital Forensic Engineering (SADFE)*, 2007, pp. 71–85.

[4] MITRE, "Indicator Removal, Technique T1070," MITRE ATT&CK. [Online]. Available: https://attack.mitre.org/techniques/T1070/

[5] X. Ding and H. Zou, "Reliable time based forensics in NTFS," School of Software, Shanghai Jiao Tong University, 2010.

[6] A. Gungor, "Date forgery analysis and timestamp resolution," Meridian Discovery, 2014.

[7] S. Y. Willassen, "Finding evidence of antedating in digital investigations," in *Proc. 3rd Int. Conf. Availability, Reliability and Security (ARES)*, 2008, pp. 26–32.

[8] E. Casey, "Digital stratigraphy: Contextual analysis of file system traces in forensic science," *J. Forensic Sci.*, vol. 63, no. 5, pp. 1383–1391, 2018.

[9] E. Zimmerman, "Eric Zimmerman's tools." [Online]. Available: https://ericzimmerman.github.io/

[10] R. Andrade, "Expose evidence of timestomping with the NTFS timestamp mismatch artifact," Magnet Forensics, 2020.

[11] J. Oh, S. Lee, and H. Hwang, "Forensic detection of timestamp manipulation for digital forensic investigation," *IEEE Access*, 2024.

[12] M. Galhuber and R. Luh, "Time for truth: Forensic analysis of NTFS timestamps," in *Proc. 16th Int. Conf. Availability, Reliability and Security (ARES)*, 2021, pp. 1–10.